Converse Corner

Community Skill Exchange Platform

PROJECT REPORT

BACHELOR OF SCIENCE (COMPUTER SCIENCE) BY

Mohammad Mohtasham Khan 6030

Under The Esteemed Guidance Of

Prof. Neha Momin

DEPARTMENT OF COMPUTER SCIENCE

MAHARASHTRA COLLEGE OF ARTS SCIENCE & COMMERCE

Affliated To University Of Mumbai

MUMBAI – 400008 MAHARASHTRA

2025-2026

PNR No :- …….. Roll No:- 6030

1. **Name of the Students :-**

_MOHAMMAD MOHTASHAM KHAN_____

2. **Title of the project :-**

Converse corner : community skill exchange platform

3. **Name of the Guide :-**

Prof. Neha Miss_____

4. **Teaching experience of the Guide :-**

Excellent_____

5. **Is this your first submission? YES** ☐ **NO** ☐

Signature of the Students :- ……

Signature of the Guide :- ……..

Signature of the Coordinator :- ……

Date :-

MAHARASHTRA COLLEGE OF ART, SCIENCE & COMMERCE

(Affiliated to University of Mumbai)

MUMBAI-MAHARASHTRA-400008

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that project entitled, "**CONVERSE CORNER: Community Skill Exchange Platform** ", is a bonafide work of Mohammad Mohtasham Khan, bearing Seat No: 6030 submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE (COMPUTER SCIENCE) From University of Mumbai.

Internal Guide **External Guide**

College Seal

ABSTRACT

CONVERSE CORNER: *is a community-driven, web-based skill exchange platform designed to make essential skill development affordable, accessible, and interactive. traditional training methods are often expensive, geographically restricted, and lack the real-time feedback that learners need to improve. to overcome these limitations, the proposed system provides a peer-to-peer environment where users can both offer and request skills, enabling collaborative growth and knowledge sharing.*

the platform is built using a lightweight and modern technology stack — flask (python) for backend, html, css, javascript with bootstrap for the frontend, and sqlite for database management. the initial system (mvp) includes key functions such as:

user registration and authentication

profile creation with skill listings
offering and requesting skills, and
an internal messaging module to facilitate coordination between users

in the future, converse corner can be extended with advanced features such as ai-based performance feedback, certification modules, and an enterprise version for corporate training. this project demonstrates the potential of technology-driven, community-based platforms in democratizing continuous personal and professional development.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **my project guide, faculty members, and mentors** for their continuous guidance, encouragement, and valuable feedback throughout the development of this project titled "**converse corner:** community skill exchange platform." their insights and support have been instrumental in shaping this work.
i am also thankful to my **friends and classmates** for their cooperation and constructive suggestions, which motivated me to improve the quality of the project.
finally, i owe my deepest appreciation to **my family** for their constant encouragement, patience, and unwavering support, which enabled me to complete this project successfully.

DECLARATION

I hereby declare that the project work entitled "**Converse Corner**: community skill exchange platform" is carried out by me as part of my academic requirements. this project report is my original work and has not been submitted, in part or in full, for the award of any other degree, diploma, or certificate.

all information, references, and resources used in this report have been duly acknowledged. the results, findings, and conclusions presented are based on my own research, analysis, and system development.

i further declare that this project is the outcome of my individual effort under the guidance of my faculty and mentors, and it meets the requirements specified by the curriculum.

Name & Signature Of The Student

MOHAMMAD MOHTASHAM

6030

TABLE OF CONTENTS

Serial No.	Contents	Page no.
1	**Chapter 1: Introduction**	**10 – 11**
1.1	Project Purpose	
1.2	Project Scope	
1.3	Project Objectives	
1.4	Project Goals	
2	**Chapter 2: Literature Review & Market Study**	**12 – 14**
2.1	Industry Overview	
2.2	Competitor Analysis	
2.3	Target Audience & Demographics	
3	**Chapter 3: System Requirement Specifications**	**15 – 17**
3.1	Product Perspective	
3.2	Product Functions	
3.3	Functional & Non-Functional Requirements	
3.4	Assumptions & Dependencies	
4	**Chapter 4: System Design**	**18 – 24**
4.1	Class Diagram	
4.2	Use-Case Diagram: Admin Use-Case	
4.3	Sequence Diagram	
4.4	ER Diagram	

4.5	Deployment Diagram	
5	**Chapter 5 : Screen-Shots Of The Website**	**25 – 28**
6 6.1 6.2	**Chapter 6 : Codes** Frontend Codes Backend Codes	**29 – 34**
7	**Chapter 7: User Interface (UI) Screens** 7.1 Login 7.2 Add Skill 7.3 Browse Skill 7.4 Message 7.5 summary	**35 – 39**
8 8.1 8.2	**Chapter 8: Implementation** Development Tools & Technologies Deployment Strategy	**40 – 43**
9 9.1 9.2 9.3 9.4	**Chapter 9: Testing** Test Cases & System Testing Unit Testing Module Testing Acceptance Testing	**44 – 47**
10 10.1 10.2 10.3	**Chapter 10: Results & Discussion** Key Results Achieved Discussion and Validation Conclusion of Results	**48 – 51**

11	**Chapter 11: Conclusion**	**52 – 53**
11.1	Summary of Work	
11.2	Key Contributions	
11.3	Final Remarks	
12	**Chapter 12: References & Bibliography**	**54 – 55**

CHAPTER – 1: Introduction

1.1 Project Purpose

The purpose of the **Converse Corner: Community Skill Exchange Platform** project is to provide a **digital, web-based platform** that simplifies and streamlines the process of skill sharing and learning. It aims to replace informal, unstructured, or costly learning methods with an **efficient, interactive, and user-friendly system** that caters to the needs of learners, mentors, and administrators. The platform facilitates the **listing, offering, and requesting of skills**, enables peer-to-peer interactions, and allows users to track their learning and teaching contributions effectively. By providing **real-time communication, skill tracking, and session management**, Converse Corner enhances collaboration, motivates active participation, and supports continuous personal and professional development. The system ensures a **low-stakes, inclusive environment** where users can practice, learn, and provide feedback without geographical or financial barriers. Overall, the project's purpose is to **democratize skill development, encourage knowledge sharing, and build a supportive global learning community**.

1.2 Project Scope

The scope of the **Converse Corner: Community Skill Exchange Platform** project is to develop a **web-based platform** that enables users to **offer, request, and exchange skills** in a collaborative community. It includes three main user roles: **Admin**, **Mentor**, and **Learner**. Admins manage users and monitor activities, Mentors list skills and provide guidance, and Learners request skills and track their progress.

The system supports **secure login, role-based access, profile management, messaging, and session coordination**. It focuses on delivering a **scalable, interactive, and user-friendly solution** for skill development, fostering collaboration, knowledge sharing, and continuous personal growth.

1.3 Project Objectives

The objectives of the **Converse Corner: Community Skill Exchange Platform** project are:

- To develop a **secure, web-based platform** that facilitates efficient peer-to-peer skill exchange.
- To implement **role-based access control**, ensuring that Admins, Mentors, and Learners have appropriate permissions for their respective tasks.
- To enable users to **create, update, and manage their profiles**, including skills they can offer and skills they wish to learn.
- To provide **real-time access** for learners to request skills and schedule learning sessions with mentors.
- To support mentors in **sharing knowledge and providing feedback** on learners' progress.
- To generate **comprehensive tracking of skill exchanges and contributions**, motivating active participation.
- To improve **collaboration, engagement, and transparency** within the community.

1.4 Project Goals

The goals of the **Converse Corner: Community Skill Exchange Platform** project are:

- To build an **intuitive and responsive web application** accessible by Admins, Mentors, and Learners across devices.
- To establish a **secure authentication and authorization framework** to protect user data and skill exchange records.
- To ensure **data accuracy and consistency** through validations and relational database management using SQLite.
- To provide **comprehensive management features** for adding, editing, viewing, and tracking users, skills, and skill exchange sessions.

- To enable **seamless communication** between mentors and learners through an internal messaging system.
- To **reduce barriers to skill development** by automating session scheduling, skill tracking, and progress reporting.
- To incorporate a **scalable architecture** supporting future growth, additional features, and AI-based feedback or certification modules.
- To enhance **community engagement, collaboration, and knowledge sharing**, promoting continuous personal and professional growth.

CHAPTER – 2 : Literature Review & Market Study

2.1 Industry Overview

The **online learning and skill development industry** has experienced rapid growth with the increasing adoption of digital platforms that enable accessible, flexible, and interactive learning. Traditional skill-building methods, such as classroom training and workshops, often involve high costs, geographic limitations, and limited opportunities for peer interaction. Community-driven platforms and web-based skill exchange systems are emerging to address these challenges by providing **peer-to-peer learning, mentoring, and practical experience** in a scalable and cost-effective manner.

The industry is moving toward platforms that facilitate **real-time communication, personalized skill matching, and tracking of learning outcomes**. Features such as skill listing, mentorship, messaging, and session scheduling are becoming standard, allowing users to **learn from peers, share expertise, and gain practical experience** outside formal institutions. Additionally, the integration of **AI feedback, certification modules, and gamification** is becoming a trend, improving user engagement, skill assessment, and motivation.

With a growing emphasis on **continuous personal and professional development**, platforms like Converse Corner are positioned to meet the

demand for **accessible, interactive, and community-driven learning ecosystems**. These platforms enable users to **connect globally, practice skills in a low-stakes environment, and collaborate with peers**, making skill development more inclusive, efficient, and engaging.

2.2 Competitor Analysis

The online skill development and peer-to-peer learning space has several established players, including **Skillshare, Coursera, Udemy, LinkedIn Learning, and local community learning platforms**. While these platforms provide structured courses and professional training, they often come with **high subscription costs, limited personalized mentorship, and minimal peer-to-peer interaction**.

In contrast, **Converse Corner** focuses on **direct skill exchange between users**, enabling a more interactive, collaborative, and low-cost approach. Unlike traditional platforms, it emphasizes **mentorship, live practice sessions, and feedback in a community-driven environment**, making skill learning more practical and socially engaging.

2.3 Target Audience & Demographics

Great! Here's the **rewritten version of 2.2 Competitor Analysis and 2.3 Target Audience & Demographics** for your **Converse Corner** project:

2.2 Competitor Analysis

The online skill development and peer-to-peer learning space has several established players, including **Skillshare, Coursera, Udemy, LinkedIn Learning, and local community learning platforms**. While these platforms provide structured courses and professional training, they often come with **high subscription costs, limited personalized mentorship, and minimal peer-to-peer interaction**.

In contrast, **Converse Corner** focuses on **direct skill exchange between users**, enabling a more interactive, collaborative, and low-cost approach. Unlike traditional platforms, it emphasizes **mentorship, live practice sessions, and feedback in a community-driven environment**, making skill learning more practical and socially engaging.

Key differentiators of Converse Corner compared to competitors include:

- **Peer-to-peer skill exchange:** Users can both teach and learn, unlike purely course-based platforms.
- **Internal messaging and session coordination:** Direct communication enables real-time practice and feedback.
- **Low-cost or free access:** Reduces financial barriers to skill development.
- **Incentive system:** Tracks contributions and encourages active participation.
- **Scalability and future growth potential:** Integration of AI feedback, certifications, and enterprise modules.

These unique features position Converse Corner as a **community-focused alternative** in the skill development ecosystem, bridging gaps left by conventional e-learning platforms.

2.3 Target Audience & Demographics

The target audience for **Converse Corner** includes a wide range of individuals seeking skill development opportunities in a flexible and interactive environment.

Primary Users:

- **Students:** Looking to improve soft skills such as communication, public speaking, and interview preparation.

- **Professionals:** Seeking career growth, upskilling, or mentorship opportunities.
- **Mentors / Skill Experts:** Individuals willing to share expertise, provide guidance, and gain teaching experience.

Demographics:

- Age group: **16–40 years**, covering high school, college, and early career professionals.
- Geographic reach: **Global**, with a focus on internet-accessible regions.
- Background: Users from **diverse educational and professional backgrounds**, including students, freelancers, young professionals, and lifelong learners.

By targeting a **broad, digitally connected audience**, Converse Corner aims to create a **vibrant, inclusive, and collaborative community**, enabling users to learn, teach, and grow together.

CHAPTER – 3: System Requirement Specifications

3.1 Product Perspective

The **Converse Corner** platform is a **web-based, community-driven skill exchange system** designed to connect learners and mentors globally. It serves as a **centralized hub** for managing users, skill offerings, skill requests, and communication between participants. The platform complements existing informal learning methods by providing a **structured, interactive, and scalable solution** for skill development.

Technically, the platform follows a **client-server architecture**, where the backend is powered by **Python with Flask**, handling business logic, data management, and server-side processing. The frontend is built using **HTML, CSS, JavaScript, and Bootstrap**, providing a **responsive and user-friendly interface** across devices. **SQLite** is used as the relational database management system to store and retrieve user profiles, skill listings, skill exchange records, and messaging data.

The system is designed to **integrate seamlessly with future enhancements**, such as AI-powered feedback, certification modules, and enterprise versions, making it scalable for larger user bases.

3.2 Product Functions

The primary functions of the **Converse Corner** platform include:

1. **User Management:** Registration, login, profile creation, editing, and secure authentication.
2. **Skill Listing:** Mentors can list skills they can teach, and learners can indicate skills they wish to acquire.
3. **Skill Exchange Requests:** Users can request sessions for skill practice and teaching.
4. **Internal Messaging:** Facilitates direct communication between mentors and learners for coordination and feedback.
5. **Session Management:** Scheduling, tracking, and confirming skill exchange sessions.
6. **Contribution Tracking:** Monitors teaching and learning activities to encourage engagement.
7. **Admin Functions:** Manage users, oversee platform activity, approve exchanges, and maintain data integrity.
8. **Notifications:** Alerts users about requests, session confirmations, and community updates.

3.3 Functional & Non-Functional Requirements

Functional Requirements:

- Secure user registration and login system.
- Role-based access control for Admins, Mentors, and Learners.
- Add, edit, and delete user profiles and skill listings.
- Request and approve skill exchange sessions.
- Messaging module for real-time communication.

- Track user contributions and provide incentives.
- Notifications for session updates and messages.

Non-Functional Requirements:

- Performance: Fast response times and smooth interactions, even with multiple simultaneous users.
- Usability: Intuitive and responsive interface for seamless navigation on desktop and mobile devices.
- Security: Protection of user data through encrypted authentication and secure storage.
- Scalability: Capability to handle an increasing number of users and future features.
- Reliability: High system uptime with minimal downtime or errors.
- Maintainability: Modular architecture for easy updates, debugging, and enhancements.

3.4 Assumptions & Dependencies

- Users have access to internet-enabled devices (desktop, laptop, or mobile).
- Users possess basic digital literacy to navigate the platform.
- System depends on Python (Flask), SQLite, HTML, CSS, JavaScript, and Bootstrap.
- Email or messaging services for notifications are reliably available.
- Future AI-based feedback and certification modules may require additional computational resources.
- The platform assumes a peer-to-peer cooperative model, where users actively participate in teaching and learning.

Dependencies

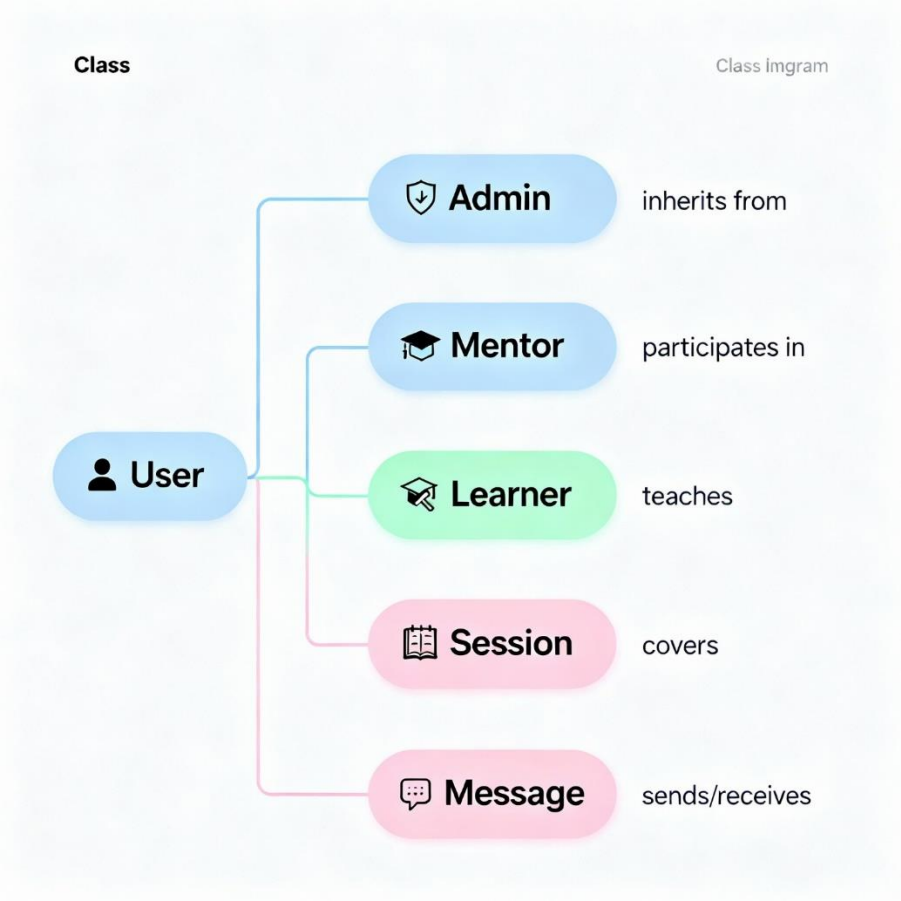
The **Converse Corner: Community Skill Exchange Platform** relies on several **software, hardware, and operational dependencies** to function efficiently:

Software Dependencies:

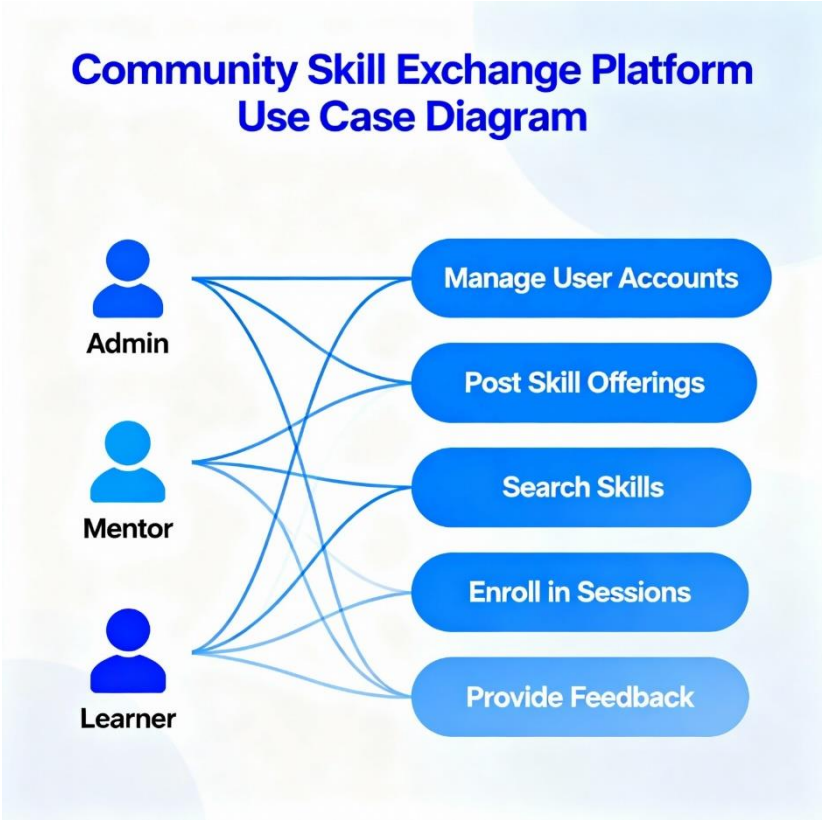
1. **Python 3.x** – Required for backend development using Flask.
2. **Flask Framework** – Handles server-side logic, routing, and API endpoints.
3. **SQLite** – Lightweight relational database for storing user profiles, skills, exchanges, and messages.
4. **HTML, CSS, JavaScript** – Frontend technologies for creating responsive and interactive user interfaces.
5. **Bootstrap** – CSS framework for mobile-friendly and consistent design.
6. **Jinja2 Templates** – For dynamic rendering of HTML pages with Flask.
7. **Python Libraries** – Additional libraries such as Flask-WTF for forms, SQLAlchemy for ORM, and Flask-Login for authentication.

CHAPTER – 4: Diagrams

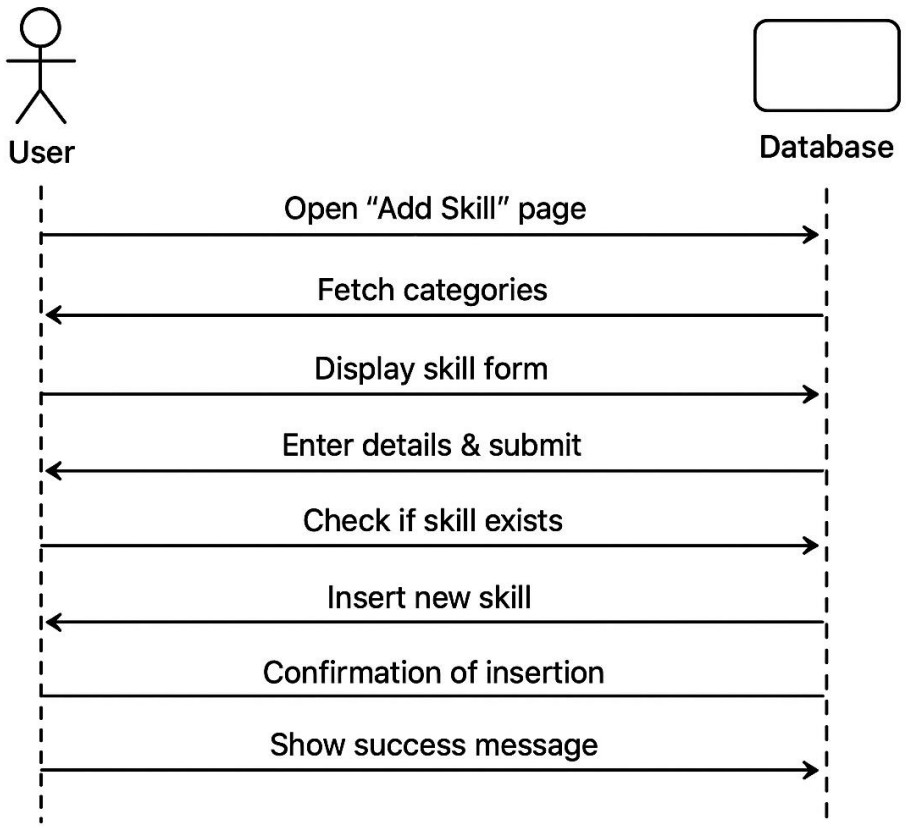
4.1 Class Diagram



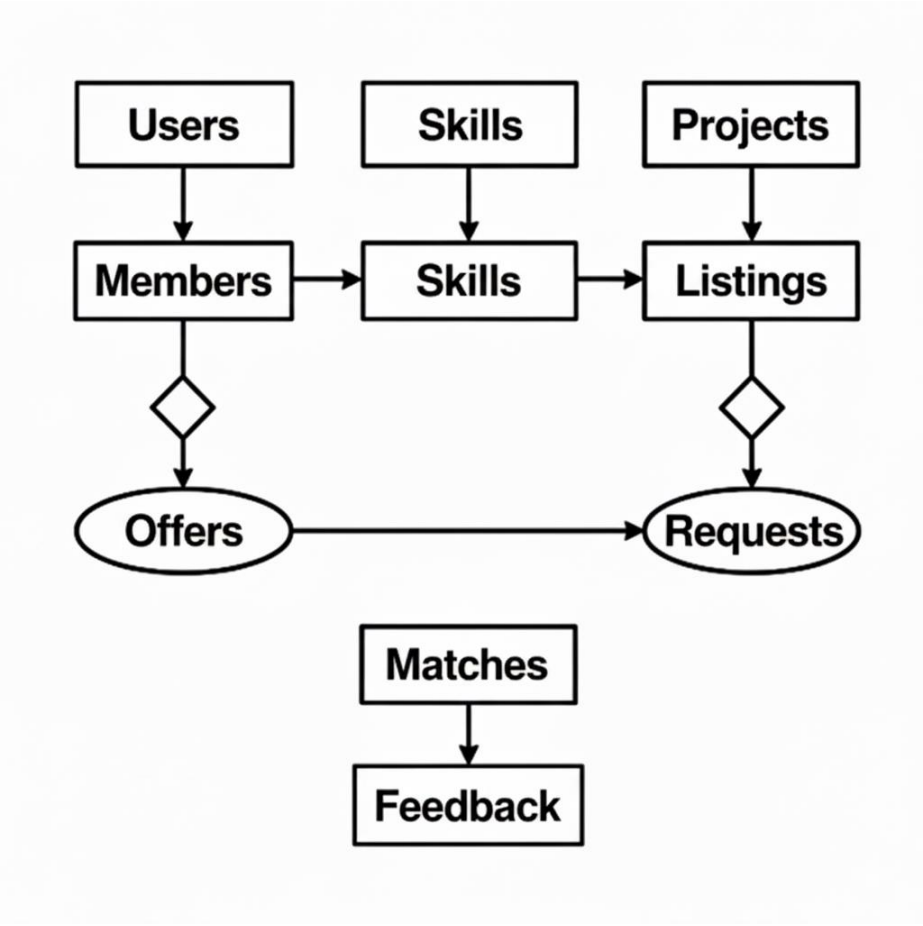
4.2 Use-Case Diagram

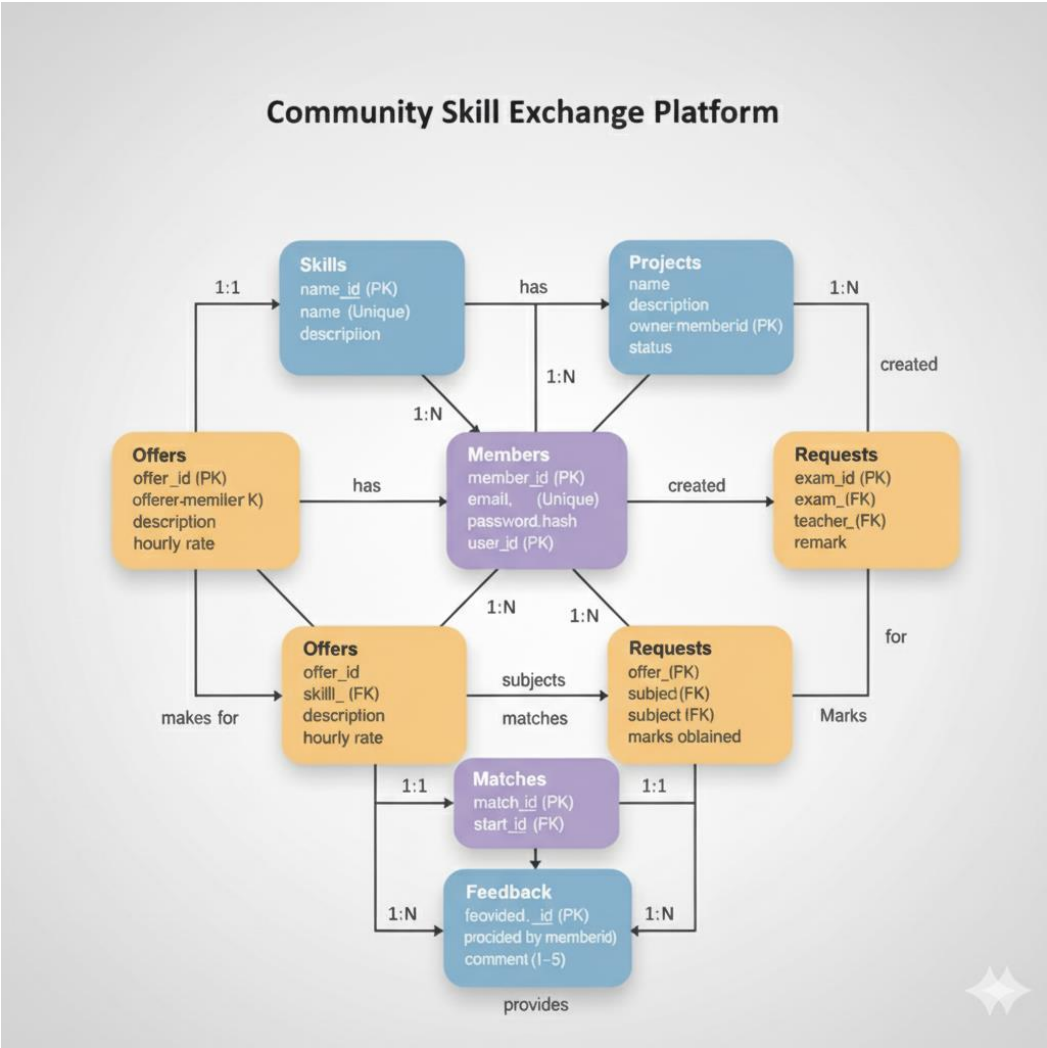


4.3 Sequence Diagram

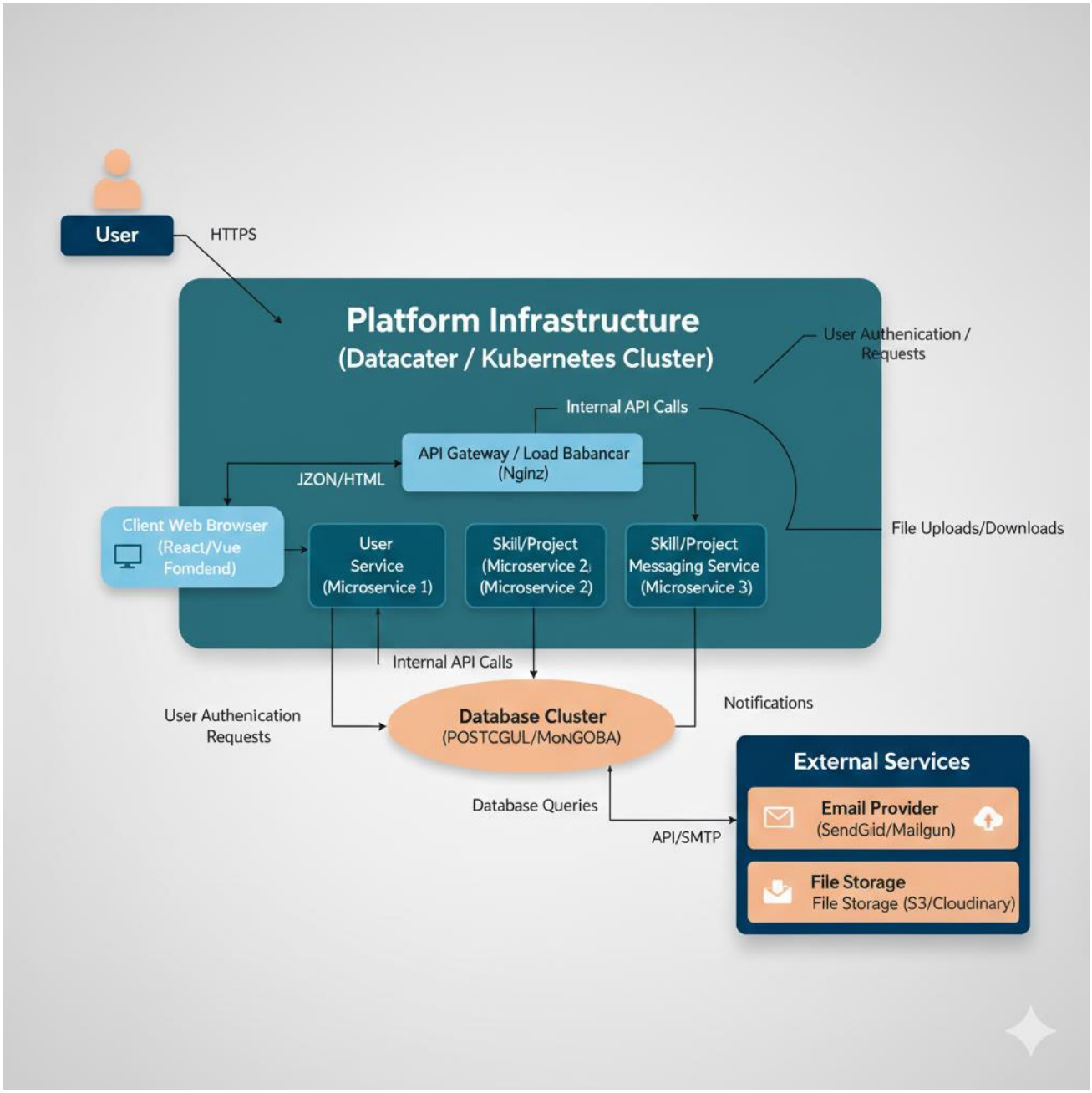


4.4 ER Diagram



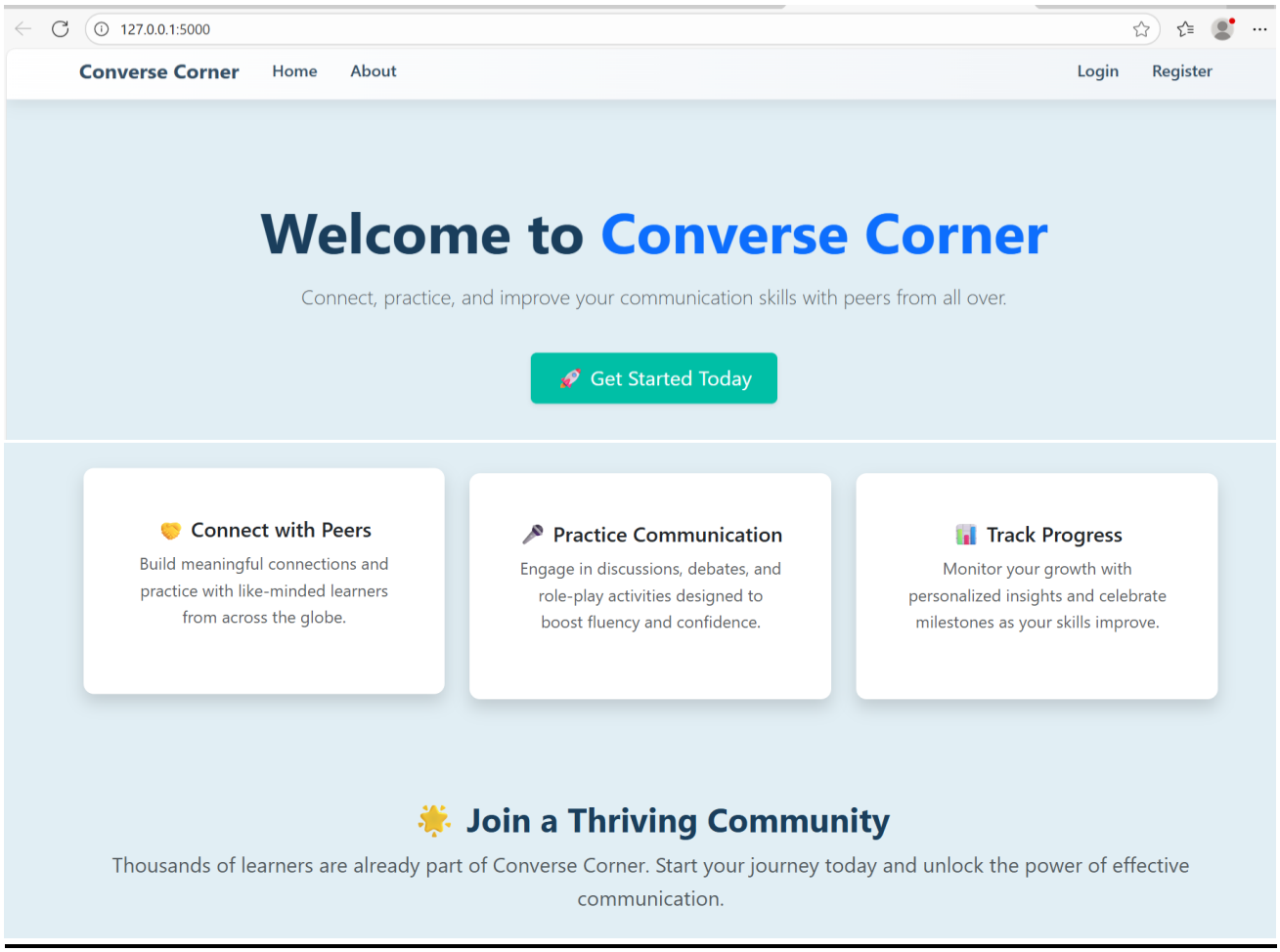


4.5 Deployment Diagram

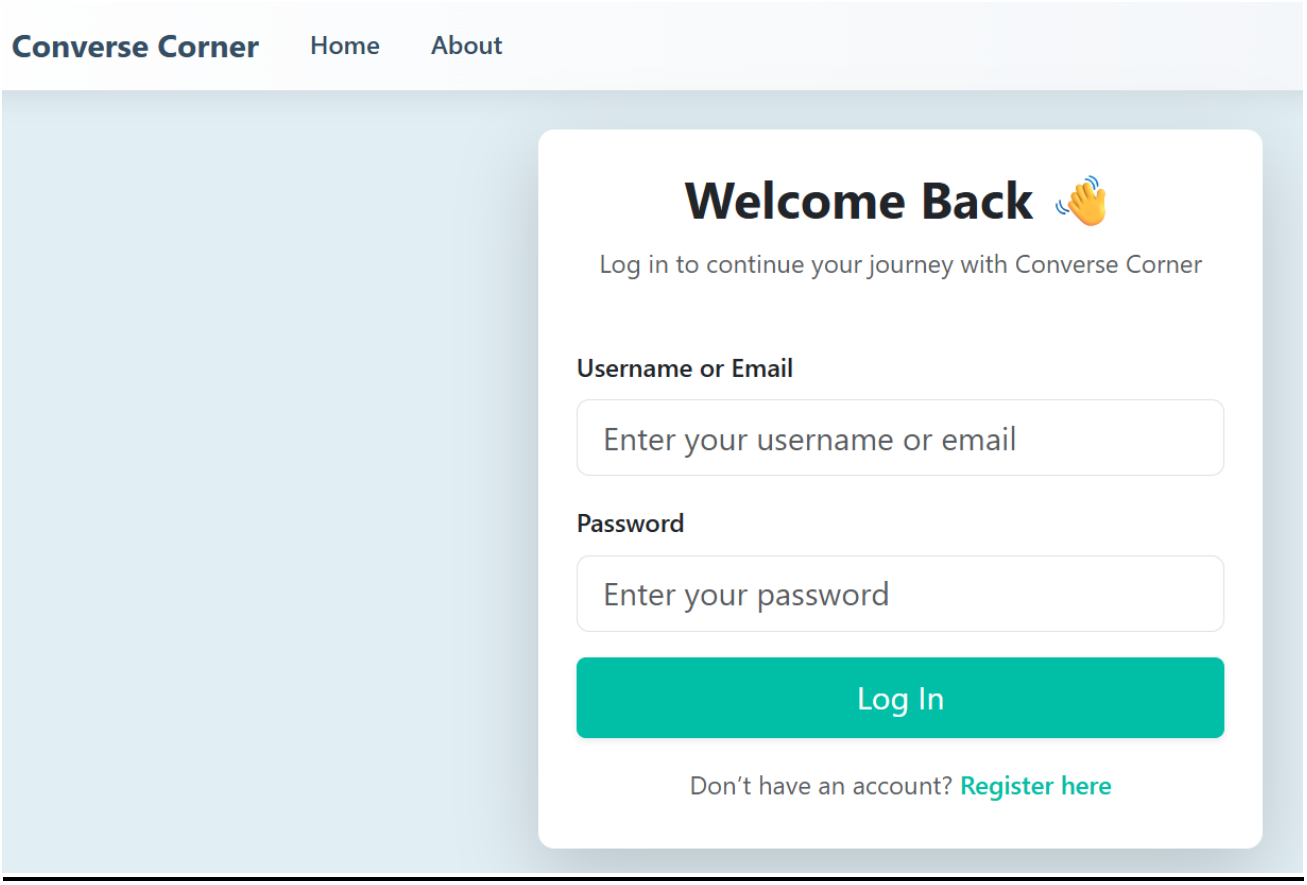

User
HTTPS
Platform Infrastructure
(Datacater / Kubernetes Cluster)
User Authenication /
Requests
Internal API Calls
API Gateway / Load Babancar
(Nginz)
JZON/HTML
Client Web Browser
(Reacl/Vue
Fomdend)
User
Service
(Microservice 1)
Skill/Project
(Microservice 2)
(Microservice 2)
Skill/Project
Messaging Service
(Microservice 3)
File Uploads/Downloads
Internal API Calls
User Authenication
Requests
Database Cluster
(POSTCGUL/MoNGOBA)
Notifications
External Services
Email Provider
(SendGiid/Mailgun)
File Storage
File Storage (S3/Cloudinary)
Database Queries
API/SMTP

CHAPTER – 5: ScreenShot Of The Website

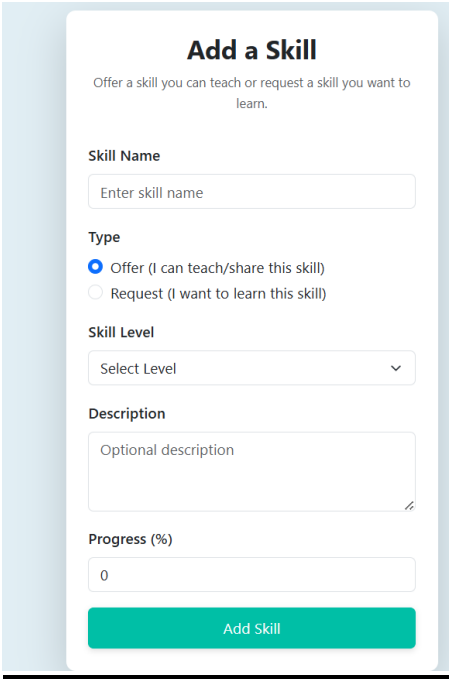
HOME PAGE



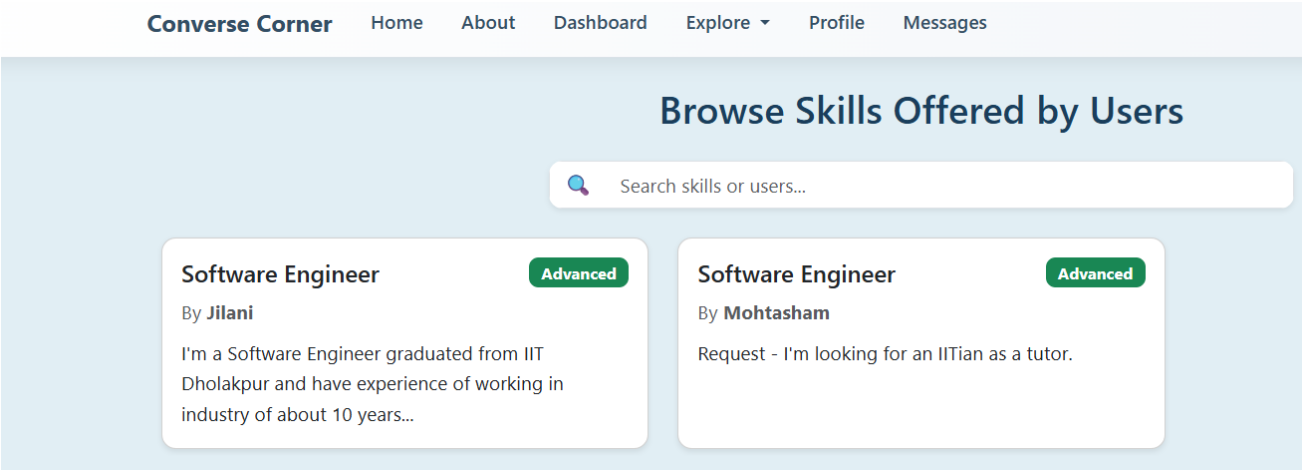
LOGIN PAGE



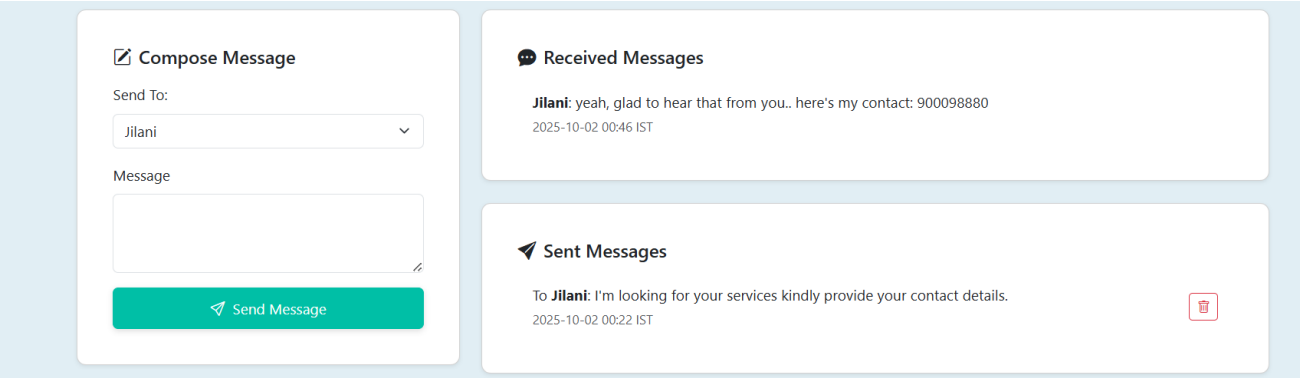
ADD SKILL



BROWSE SKILL



REQUEST / MESSAGE SKILL



CHAPTER – 6: CODES

-> Frontend

1. HTML Structure Snippets

Base.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1" />
   <title>{% block title %}Converse Corner{% endblock %}</title>

   <!-- Bootstrap CSS CDN -->
   <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.1/font/bootstrap-icons.css">

   <!-- Custom CSS -->
   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}" />

   {% block head %}{% endblock %}
</head>
<body>
   <nav class="navbar navbar-expand-lg navbar-light bg-light mb-4">
      <div class="container">
         <!-- Brand -->
         {% if current_user.is_authenticated %}
            <a class="navbar-brand" href="{{ url_for('dashboard') }}">Converse
Corner</a>
         {% else %}
            <a class="navbar-brand" href="{{ url_for('home_page') }}">Converse
Corner</a>
         {% endif %}

         <!-- Toggler -->
         <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarContent"
```

```
            aria-controls="navbarContent" aria-expanded="false" aria-label="Toggle
navigation">
            <span class="navbar-toggler-icon"></span>
        </button>

        <!-- Navbar links -->
        <div class="collapse navbar-collapse" id="navbarContent">
            <ul class="navbar-nav me-auto mb-2 mb-lg-0">
                <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('home_page') }}">Home</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('about_page') }}">About</a>
                </li>

                {% if current_user.is_authenticated %}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ url_for('dashboard')
}}">Dashboard</a>
                    </li>

                    <!-- Explore Dropdown -->
                    <li class="nav-item dropdown">
                        <a class="nav-link dropdown-toggle" href="#"
id="exploreDropdown" role="button" data-bs-toggle="dropdown" aria-
expanded="false">
                            Explore
                        </a>
                        <ul class="dropdown-menu" aria-labelledby="exploreDropdown">
                            <li><a class="dropdown-item" href="{{ url_for('add_skill')
}}">Add Skill</a></li>
                            <li><a class="dropdown-item" href="{{ url_for('browse_skill')
}}">Browse Skills</a></li>
                        </ul>
                    </li>

                    <li class="nav-item">
                        <a class="nav-link" href="{{ url_for('profile') }}">Profile</a>
                    </li>
                    <li class="nav-item">
```

```
                    <a class="nav-link" href="{{ url_for('messages')
}}">Messages</a>
                  </li>
                {% endif %}
            </ul>

            <!-- Right side: user greeting / auth links -->
            <ul class="navbar-nav ms-auto mb-2 mb-lg-0">
                {% if current_user.is_authenticated %}
                  <li class="nav-item d-flex align-items-center">
                    <span class="navbar-text me-3">Hello, {{ current_user.username
}}!</span>
                  </li>
                  <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('logout') }}"
                      onclick="return confirm('Are you sure you want to log out?');">
                      Logout
                    </a>
                  </li>
                {% else %}
                  <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('login') }}">Login</a>
                  </li>
                  <li class="nav-item">
                    <a class="nav-link" href="{{ url_for('register') }}">Register</a>
                  </li>
                {% endif %}
            </ul>
          </div>
      </div>
   </nav>

   <main class="container">
      {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
          {% for category, message in messages %}
            <div class="alert alert-{{ category }} alert-dismissible fade show"
role="alert">
              {{ message }}
```

```
            <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
            </div>
          {% endfor %}
        {% endif %}
      {% endwith %}

      {% block content %}{% endblock %}
   </main>

   <!-- Bootstrap JS Bundle -->
   <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"><
/script>
   {% block scripts %}{% endblock %}
</body>
</html>
```

2. CSS Styling Snippets
Style.css

```
:root {
   /* Primary & Secondary */
   --primary-color: #1A3E5C;       /* Deep navy blue */
   /* Neutrals */
   --neutral-dark: #334E65;          /* Dark gray/blue for primary elements
*/
   --text-main: #1A3E5C;             /* Main body text color */
   --text-light: #6C7A89;           /* Secondary/lighter dark text color */
}

/* Base Body Styling */
body {
```

```
    color: var(--text-main); /* Sets main text to #1A3E5C */
}

/* Dark Navbar - Premium (Sets dark background) */
.navbar-dark {
    background: linear-gradient(90deg, #1A3E5C 0%, #1F4F70 100%);
}

/* Light Navbar - Premium (Sets dark text color) */
.navbar-light .navbar-brand {
    color: var(--neutral-dark); /* Sets brand text to #334E65 */
}

.navbar-light .nav-link {
    color: var(--neutral-dark); /* Sets link text to #334E65 */
}

/* Alerts */
.alert-info { background: var(--neutral-dark); color: var(--primary-contrast); border: none; }
.alert-warning { background: var(--warning); color: #222; border: none; }
/* Sets warning text to near-black (#222) */
```

Backend (app.py)

```
# --------------------- CRITICAL IMPORTS & CONFIG

from datetime import datetime, timedelta, timezone
from flask import abort
from flask_login import LoginManager, login_user, logout_user, login_required, current_user, UserMixin
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy
```

```
import threading, webbrowser, time

# Initialize Flask app and configure database and secret key
app = Flask(__name__)
app.config.update(
    SECRET_KEY='Mohtasham',
    SQLALCHEMY_DATABASE_URI='sqlite:///converse_corner.db',
    SQLALCHEMY_TRACK_MODIFICATIONS=False,
)
db = SQLAlchemy(app)

login_manager = LoginManager(app)
login_manager.login_view = "login"

IST_OFFSET = timedelta(hours=5, minutes=30)

# ---------------- User Model ----------------
class User(db.Model, UserMixin):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)

    skills = db.relationship('Skill', backref='user', lazy=True)

    # Only define relationships without backrefs
    sent_messages = db.relationship('Message', foreign_keys='Message.sender_id', lazy=True)
    received_messages = db.relationship('Message', foreign_keys='Message.receiver_id', lazy=True)

# ---------------- Skill Model ----------------
class Skill(db.Model):
```

```
    __tablename__ = 'skills'  # optional explicit table name

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    level = db.Column(db.String(50))
    description = db.Column(db.Text)
    progress = db.Column(db.Integer, default=0)  # 0 to 100
    last_updated = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # ForeignKey to User
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)

    def __repr__(self):
        return f"<Skill {self.name} ({self.progress}%)>"

# ---------------- Message Model ----------------
class Message(db.Model):
    __tablename__ = 'messages'

    id = db.Column(db.Integer, primary_key=True)
    sender_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    receiver_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    content = db.Column(db.Text, nullable=False)
    timestamp = db.Column(db.DateTime, default=datetime.utcnow)
    is_read = db.Column(db.Boolean, default=False)

    sender = db.relationship('User', foreign_keys=[sender_id], backref='messages_sent')
    receiver = db.relationship('User', foreign_keys=[receiver_id], backref='messages_received')
```

```
    @property
    def ist_timestamp(self):
        return self.timestamp + timedelta(hours=5, minutes=30)

__
#----- Routes----------
# Route for home page rendering index.html
@app.route('/')
def home():
    return render_template('index.html')

# Route to handle user registration (GET shows form, POST processes form)
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        u, e, p, cp = (request.form[k] for k in ('username', 'email', 'password', 'confirm_password'))
        if p != cp:
            flash('Passwords do not match!')
            return redirect(url_for('register'))

        # Check if username or email already exists
        user_exists = User.query.filter((User.username == u) | (User.email == e)).first()
        if user_exists:
            flash('Username or email already registered. Please use another.')
            return redirect(url_for('register'))

        new_user = User(username=u, email=e, password=p)
        db.session.add(new_user)
        db.session.commit()
        flash('Registration successful. Please log in.')
        return redirect(url_for('login'))

    return render_template('register.html')
```

```
# Login Manager
@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

# Route for user login page (logic to be added)
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username_or_email = request.form['username']
        password = request.form['password']
            user = User.query.filter((User.username == username_or_email) |
(User.email == username_or_email)).first()
            if user and user.password == password:  # Replace with hashed
password check in production
                login_user(user)
                flash('Logged in successfully.')
                return redirect(url_for('dashboard'))  # Create dashboard route
        else:
                flash('Invalid username/email or password.')
                return redirect(url_for('login'))
    return render_template('login.html')

@app.route('/logout')
@login_required
def logout():
    logout_user()
    flash('You have been logged out.')
    return redirect(url_for('login'))

@app.route('/home')
def home_page():
    return render_template('home.html')
```

```
# Route to add a new skill and update skill
@app.route('/add_skill', methods=['GET', 'POST'])
@login_required
def add_skill():
    if request.method == 'POST':
        # Get form data
        name = request.form['skill_name']
        level = request.form['skill_level']
        description = request.form.get('skill_description', '')
        progress = request.form.get('progress', 0)
        skill_type = request.form.get('skill_type', 'Offer')  # Default to 'Offer'

        try:
            progress = int(progress)
            if progress < 0 or progress > 100:
                flash("Progress must be between 0 and 100.", "warning")
                return redirect(url_for('add_skill'))
        except ValueError:
            flash("Progress must be a number.", "warning")
            return redirect(url_for('add_skill'))

        # Create new Skill object
        new_skill = Skill(
            name=name,
            level=level,
                description=f"{skill_type} - {description}" if description else
skill_type,
            progress=progress,
            user_id=current_user.id
        )

        # Save to DB
        db.session.add(new_skill)
        db.session.commit()
        flash('Skill added successfully.', 'success')
```

```
        return redirect(url_for('profile'))

    return render_template('add_skill.html')

@app.route('/dashboard')
@login_required
def dashboard():
    all_messages = Message.query.filter(
        (Message.sender_id == current_user.id) | (Message.receiver_id ==
current_user.id)
    ).order_by(Message.timestamp.desc()).all()
    recent_messages = all_messages[:3]
    return render_template('dashboard.html', user=current_user,
recent_messages=recent_messages)

# Route to browse all skills
@app.route('/browse_skill')
def browse_skill():
    skills = Skill.query.all()
    return render_template('browse_skill.html', skills=skills)

# Route to view user profile with their skills (protected)
@app.route('/profile')
@login_required
def profile():
    user = current_user
    return render_template("profile.html", user=user)

@app.route('/about')
def about_page():
    return render_template('about.html')

# Route to compose a message
@app.route('/messages', methods=['GET', 'POST'])
@login_required
```

```
def messages():
    all_messages = Message.query.filter(
        (Message.sender_id == current_user.id) | (Message.receiver_id == current_user.id)
    ).order_by(Message.timestamp.desc()).all()

    sent = [msg for msg in all_messages if msg.sender_id == current_user.id]
    received = [msg for msg in all_messages if msg.receiver_id == current_user.id]

    recipients = User.query.filter(User.id != current_user.id).all()

    if request.method == 'POST':
        receiver_id = int(request.form['receiver_id'])
        content = request.form['content'].strip()
        if not content:
            flash("Message cannot be empty!", "warning")
            return redirect(url_for('messages'))

        receiver = User.query.get_or_404(receiver_id)
        msg = Message(sender_id=current_user.id, receiver_id=receiver_id, content=content)
        db.session.add(msg)
        db.session.commit()
        flash(f"Message sent to {receiver.username}!", "success")
        return redirect(url_for('messages'))

    return render_template(
        'messages.html',
        sent=sent,
        received=received,
        recipients=recipients
    )

#---------Delete Route----------
```

```
@app.route('/delete_skill/<int:skill_id>', methods=['POST'])
@login_required
def delete_skill(skill_id):
    # Fetch the skill
    skill = Skill.query.get_or_404(skill_id)

    # Ensure the skill belongs to the logged-in user
    if skill.user_id != current_user.id:
        flash("You are not authorized to delete this skill.", "danger")
        return redirect(url_for('profile'))

    try:
        db.session.delete(skill)
        db.session.commit()
        flash(f"Skill '{skill.name}' deleted successfully.", "success")
    except Exception as e:
        db.session.rollback()
        flash("An error occurred while deleting the skill.", "danger")
        print(e)

    return redirect(url_for('profile'))

@app.route('/delete_message/<int:message_id>', methods=['POST'])
@login_required
def delete_message(message_id):
    msg = Message.query.get_or_404(message_id)

    # Only allow deleting if the current user is the sender
    if msg.sender_id != current_user.id:
        flash("You are not authorized to delete this message.", "danger")
        return redirect(url_for('messages'))

    try:
        db.session.delete(msg)
        db.session.commit()
```

```
        flash("Message deleted successfully.", "success")
    except Exception as e:
        db.session.rollback()
        flash("An error occurred while deleting the message.", "danger")
        print(e)

    return redirect(url_for('messages'))

if __name__ == '__main__':
    if not app.config.get('SECRET_KEY'):
        app.config['SECRET_KEY'] = 'supersecretkey'

    with app.app_context():
        db.create_all()

    url = 'http://127.0.0.1:5000'
    webbrowser.open_new(url)  # Opens browser immediately

    app.run(debug=True, use_reloader=False)
```

CHAPTER – 7: User Interface (UI) Screens

7.1 Login Module

The **Login Module** is the entry point of the **Community Skill Exchange Platform**, allowing registered users to access their personal dashboards securely. It is implemented using **Flask-Login** for authentication and **Werkzeug** for password hashing.

When the user submits the login form, the system retrieves the entered **email** and **password**, checks the user record from the database using:

user = User.query.filter_by(email=email).first()

The provided password is verified with the stored hash using:

user.check_password(password)

If the credentials are valid, login_user(user) creates a secure session, and the user is redirected to the **dashboard**. Invalid attempts display an error message prompting re-entry of credentials.

This module ensures **secure access**, **session management**, and a **smooth login experience** for users.

7.2 Add Skill

This module allows authenticated users to **add, view, and manage their skills** within the **Community Skill Exchange Platform**.
Each skill entry is securely linked to the user's account, ensuring that skill data remains personalized and easily accessible through their dashboard.

7.2.1 Adding a New Skill

The /add-skill route, accessible to all **logged-in users**, handles the process of adding a new skill to the system.

Mechanism:

Form Validation:
The system receives skill details such as *Skill Name*, *Category*, *Description*, and *Type* (Teach or Learn). Basic validation checks ensure that all required fields are properly filled.

Skill Record Creation:
A new record is created in the **Skill table**, automatically linked to the current user using the user's user_id as a foreign key.

Database Insertion:
The new skill entry is added to the database using SQLAlchemy's ORM methods.

Example:

db.session.add(new_skill)

db.session.commit()

Confirmation:
On successful insertion, a success flash message (e.g., *"Skill added successfully!"*) is displayed, and the user is redirected back to their **Dashboard** to view the updated list of skills.

7.2.2 Deleting a Skill

The /delete-skill/<int:skill_id> route allows users to delete their previously added skills.

Mechanism:

Access Control:
Only the **owner of the skill** (verified via session user ID) can delete that skill. Unauthorized access is restricted.

Cascade Deletion:
The system ensures that when a skill is deleted, any **related records** (such as messages or exchanges linked to that skill) are also removed to maintain data integrity.

Confirmation:
After deletion, the user receives a flash message (*"Skill deleted successfully"*), and the dashboard refreshes with the updated list.

This module enables smooth and secure **skill management**, ensuring each user can **easily maintain their offered and requested skills** while preserving database integrity.

7.3 Browse Skill

The **Browse Skill** module allows users to explore and discover skills shared by other members of the community. It acts as the core interaction point of **Converse Corner**, where users can find people to learn from or collaborate with.

Functionality

- Users can **view all available skills** posted by others, categorized as "Teach" or "Learn."
- Each skill card displays key details such as the **Skill Name**, **Description**, **Category**, and the **User** offering it.
- The interface includes a **search bar and filters** to make browsing faster and more user-friendly.
- Logged-in users can express interest in a skill by **sending a connection request** or **starting a conversation** with the skill owner.

Backend Integration

- The data for skills is dynamically fetched from the **database (SQLite via SQLAlchemy)**.
- Filtering and search are handled through **Flask routes**, which query the database based on user input.
- Only **active users and verified skills** are displayed, maintaining quality and authenticity in the skill listings.
- This module allows authorized users of Converse Corner to create and manage skill sessions, which represent organized learning or exchange events for specific skills.

7.4 Message

The **Message** module enables seamless communication between users within **Converse Corner**. It serves as the primary interaction feature that allows members to exchange information, discuss skills, and coordinate learning sessions.

Functionality

- Users can **send and receive messages** in real time after connecting through a skill post.
- Each conversation is stored in the database and linked to both participating users.
- A **simple chat-style interface** displays messages chronologically for easy readability.
- Users can **view previous conversations**, **reply instantly**, and **manage chats** through the dashboard.

Backend Integration

- Each message record is stored in the **Messages** table, containing the sender ID, receiver ID, content, and timestamp.
- The messaging route handles both **sending (POST)** and **retrieving (GET)** messages using Flask.
- SQLAlchemy relationships ensure messages are properly associated with their respective users, maintaining data consistency.

7.5 Summary

The user interface of Converse Corner is designed to be simple, clean, and responsive. Users can easily login, add and manage skills, browse for new skills, and participate in organized skill sessions. Admins and mentors have additional access for managing sessions and maintaining system integrity.

The UI emphasizes usability and clarity, ensuring a smooth experience across desktop and mobile devices.

CHAPTER – 8: Implementation

The implementation phase of Converse Corner involves converting the system design into a fully functional platform. This chapter explains the tools, technologies, coding standards, and deployment strategies used to develop the application. The goal is to ensure that the platform is robust, efficient, and scalable.

8.1 Development Tools & Technologies

The Converse Corner project was built using a modern, Python-centric technology stack, prioritizing security, simplicity, and scalability.

- Backend Framework: The application uses Flask, a lightweight micro-framework, to handle routing, request processing, and application structure. All backend logic is written in Python 3.x.
- Database & ORM: SQLAlchemy is used as the Object-Relational Mapper (ORM) to define the database schema (Users, Skills, Skill Sessions, Messages) using Python classes. This approach simplifies operations like cascade deletion. During development, SQLite is used for convenience.
- User Management & Security:
    - Flask-Login manages user sessions and tracks the currently logged-in user.
    - Werkzeug handles password hashing, ensuring that user credentials are stored securely.
    - Role-Based Access Control is enforced using a custom @roles_required decorator to segment features between regular users, mentors, and admins.
- Form Handling: Flask-WTF provides robust server-side validation for all forms, ensuring data integrity.

- Reporting & Export: The system can generate printable reports, such as skill session summaries or user skill lists, using libraries like PDFKit (dependent on wkhtmltopdf) to convert HTML pages to PDF.

This setup ensures a secure, maintainable, and efficient platform for skill sharing and learning.

8.2 Deployment Strategy

A. Environment and Hosting

The application is deployed on a Platform as a Service (PaaS), such as Render or Railway, which handles server management, networking, and SSL automatically.

- Web Server: Gunicorn runs the Flask application in production for efficient request handling.
- Database: SQLite used in development is replaced by a production-grade database, such as PostgreSQL, to ensure data persistence, scalability, and support for multiple concurrent users.

B. Deployment Process

The deployment process follows a CI/CD approach:

1. Code Preparation: The application includes a requirements.txt file listing all dependencies. Optionally, a Dockerfile can be used for environment consistency.
2. Configuration: Production-sensitive information, such as database URIs and application secrets, are stored as environment variables.
3. Git Integration: The codebase is pushed to a remote repository like GitHub.
4. Automatic Deployment: The PaaS platform detects updates, builds the application, installs dependencies, and launches it using Gunicorn.
5. Reliability: The platform ensures networking, SSL encryption, continuous health checks, and automatic service restarts for high availability.

CHAPTER – 9: TESTING

Testing is a crucial phase of any project. It ensures that the system is working as intended, all features are functional, and users have a smooth experience. In **Converse Corner**, testing focuses on both the **frontend (UI)** and **backend (database and logic)**.

9.1 Test Cases & System Testing

System testing verifies that the fully integrated system meets all functional requirements for different user roles and critical features. It ensures **data integrity**, **security**, and correct operation of key functionalities.

Key Test Scenarios

1. **Authentication and Access Control**
    - **Login Success/Failure:** Verify that registered users can log in with valid credentials and receive appropriate error messages for invalid logins.
    - **Role Restrictions:** Ensure that users cannot access features restricted to other roles. For example, a regular user cannot access admin-only routes.
2. **Data Integrity (CRUD Operations)**
    - **Skill Management:** Attempting to add duplicate skills should be blocked, and appropriate error messages displayed.
    - **Skill Updates:** Re-entering or editing a skill should update the existing record without creating duplicates.
3. **Skill Session Deletion**
    - Verify that deleting a skill session removes all associated skills and related data, ensuring no orphaned records remain.
4. **Reporting and Calculations**

- **Role-Based View:** Users see only their own skills and sessions; admins/mentors can view all.
- **Accuracy Checks:** Ensure that calculated statistics, such as skill counts or session participation, are correct.
- **PDF Generation:** Verify that reports can be exported to PDF accurately.

9.2 Unit Testing

Unit tests focus on validating the atomic functionality of utility functions and core logic independent of the web interface.

Key Areas

1. **User Authentication**
   - **Password Hashing:** Ensure passwords are securely hashed.
   - **Login Validation:** Check that the system correctly validates credentials.
2. **Skill Logic**
   - **Add/Update Skills:** Verify that adding a new skill inserts a record and editing updates the same record.
   - **Skill Session Logic:** Ensure session creation and linking to skills behaves correctly.
3. **Database Model Integrity**
   - Confirm that unique constraints on skill names or session names are enforced.
   - Check that cascade deletions work properly when sessions or skills are removed.

Methodology: Unit tests were written using Python's unittest module, with a temporary in-memory SQLite database for isolated and repeatable testing.

9.3 Module Testing (Integration Testing)

Module testing ensures that related components (routes, forms, and models) work together correctly.

Focus Areas

1. **User and Skill Management Integration**
    - Form submissions correctly validate data, create linked user and skill records, and commit transactions successfully.
2. **Skill Updates and Session Linking**
    - Re-adding or editing a skill correctly updates the database without duplication.
    - Skills can be properly associated with skill sessions.
3. **Reporting and Feedback**
    - Generating reports retrieves data from multiple tables and displays accurate results.
    - Success messages and page redirects are displayed correctly after operations.
4. **Data Deletion Integrity**
    - Deleting a skill session automatically removes linked skills and related records, maintaining database integrity.

9.4 Acceptance Testing

Acceptance testing was performed with realistic dummy data to ensure the system met functional requirements and workflows.

Scenarios

1. **User Scenarios**
    - Regular users added and updated skills, linked them to sessions, and verified that only their own data was visible.
2. **Admin/Mentor Scenarios**
    - Admins created sessions, managed all skills, and confirmed proper cascade deletions.
3. **Security Scenarios**
    - Users successfully updated passwords and verified login functionality.

Conclusion

Acceptance testing confirmed that **Converse Corner** is functional, secure, and ready for deployment. All critical workflows, role-based access, and data management features operate as intended, ensuring a smooth experience for all users.

Chapter – 10: This chapter presents the main outcomes

of the Converse Corner project, evaluating the system against the initial goals of security, data integrity, and efficient skill management and exchange. The discussion highlights how the core modules effectively address challenges in managing skills, sessions, and user interactions in a community platform.

10.1 Key Results Achieved

The development of Converse Corner successfully produced a fully functional, multi-user skill exchange platform, achieving the following key results:

10.1.1 Successful Role-Based Access Control

The platform enforces granular access control using Flask-Login and a custom @roles_required decorator:

- Admin: Has exclusive access to critical management features such as creating/deleting skill sessions and moderating content.
- Mentor/Teacher: Can manage skills and sessions, track participation, and approve exchanges.
- Regular User: Can add, update, or browse skills and participate in sessions, but cannot access administrative functions.

10.1.2 Data Integrity and Relationships

The use of SQLAlchemy and defined relationships ensures high data integrity:

- Referential Integrity: Skills, sessions, and messages are correctly linked to user accounts.
- Update Logic: Editing an existing skill updates the same record instead of creating duplicates.
- Cascade Deletion: Removing a skill session automatically deletes all linked skills and associated messages, preventing orphaned data.

10.1.3 Accurate & Comprehensive Reporting

The platform provides clear and reliable reports on user skills and session participation:

- Custom Calculations: Aggregates skill statistics, session participation, and learning/exchange metrics.
- Secure Export: Integration with PDFKit allows professional, non-editable PDF reports for sharing or archiving.

10.2 Discussion and Validation

The results demonstrate that the design and implementation decisions effectively met the project objectives:

10.2.1 Efficiency in Data Management

Database relationships simplify operations such as linking skills to sessions and tracking participation. Users can manage all aspects of their skills via a single interface, reducing workflow complexity and improving usability.

10.2.2 Security Validation

The layered security approach is effective:

- Data Security: User passwords are stored as SHA-256 hashes using Werkzeug.
- Access Control: The role-based access decorator ensures users can only access authorized functions, maintaining platform security.

10.2.3 Addressing Platform Needs

Unlike large-scale platforms, Converse Corner provides a focused, lightweight solution for community skill sharing. The streamlined Flask environment ensures easy deployment, low hosting requirements, and minimal technical overhead.

10.3 Conclusion of Results

In summary, Converse Corner successfully delivered a robust, secure, and efficient skill exchange platform. The integration of Flask, SQLAlchemy, and Flask-Login created a flexible foundation for multi-user functionality. Features like cascade deletion, update logic for skills, and accurate session reporting validate the system as a reliable tool for managing and exchanging skills in a community setting.

CHAPTER – 11: 11.1 Summary of Work

The Converse Corner project successfully developed a comprehensive, multi-user community skill exchange platform using Flask and SQLAlchemy. The main goal was to replace informal or manual skill tracking with a secure, automated, and efficient digital system for managing users, skills, and skill sessions.

Key Achievements and Modules

The project included several interconnected modules, each secured with Role-Based Access Control (RBAC):

1. User and Entity Management (Admin Focus)
    - Implemented secure routes for adding users and linking them to their accounts.
    - Developed a utility to automate password hashing and role assignment.
2. Skill and Session Management
    - Created models and routes for defining skills and organizing them into sessions.
    - Enforced data integrity by preventing duplicate skills and handling database constraints.
3. Skill Updates and Participation Tracking (User Focus)
    - Implemented logic for adding or updating skills, ensuring no duplicate entries.

- Enabled users to track participation in skill sessions and exchange activities.

4. Reporting Engine
    - Compiled data across users, skills, and sessions for clear reporting.
    - Integrated PDFKit to generate secure, professional reports on demand.

11.2 Key Contributions

The project made several notable contributions in system architecture, security, and data integrity:

1. Robust Skill and Session Management
    - The platform efficiently tracks skills, session participation, and user engagement.
    - Users can add, update, and manage skills without creating duplicate entries.
2. Comprehensive Role-Based Access Control
    - Custom @roles_required decorator ensures that only authorized users can perform sensitive actions.
    - Regular users can manage their own skills, while mentors and admins have broader access.
3. Advanced Database Integrity Mechanisms
    - Upsert Logic: Updating a skill automatically adjusts existing records to prevent duplication.
    - Cascade Deletion: Removing a session deletes all associated skills and related data, avoiding orphaned records.
4. Consolidated User and Entity Management
    - Centralized utilities link login credentials to user profiles, streamlining enrollment and access management.

11.3 Final Remarks

Converse Corner successfully transitions skill management from informal methods to a secure, digital, and efficient system.

The platform's success lies in its robust architecture, leveraging Flask for lightweight development and SQLAlchemy for powerful database modeling. Role-Based Access Control ensures security, and the Upsert logic and cascade deletions maintain data integrity.

The reporting features provide accurate, comprehensive, and secure summaries of skills and sessions, including professional PDF exports.

Overall, Converse Corner demonstrates how modern web development principles can create a scalable, secure, and user-friendly skill exchange platform, ready for deployment and future feature expansion.

CHAPTER – 12: Bibliography & References

This chapter lists the primary documentation, external libraries, and official resources utilized during the development of the **Converse Corner** project.

A. Core Framework and Language Documentation

1. **Python Language Reference**
    - Official documentation for the Python programming language.
    - Source: Python Software Foundation
    - URL: https://docs.python.org/3/
2. **Flask Documentation**
    - Official documentation for the Flask web development micro-framework.
    - Source: Pallets
    - URL: https://flask.palletsprojects.com/en/latest/
3. **SQLAlchemy Documentation**
    - Official documentation for the SQL Object Relational Mapper (ORM) used for database modeling.
    - Source: SQLAlchemy Project
    - URL: https://docs.sqlalchemy.org/en/latest/

B. Libraries and Extensions

4. **Flask-Login Documentation**
    - Official guide for managing user sessions in Flask.
    - Source: Flask-Login Community
    - URL: https://flask-login.readthedocs.io/en/latest/
5. **Flask-WTF Documentation**
    - Documentation for integration with WTForms, used for secure form handling.
    - Source: Flask-WTF Community

- URL: https://flask-wtf.readthedocs.io/en/latest/

6. **Werkzeug Documentation**
    - Documentation for the WSGI utility library, used for password hashing and security.
    - Source: Pallets
    - URL: https://werkzeug.palletsprojects.com/en/latest/
7. **pdfkit Documentation**
    - Python library for converting HTML content to PDF.
    - Source: pdfkit Community
    - URL: https://pdfkit.org/
8. **wkhtmltopdf**
    - Open-source command-line tool required by pdfkit for HTML rendering and PDF generation.
    - Source: wkhtmltopdf Project
    - URL: https://wkhtmltopdf.org/